

Universidade Federal do Ceará

Campus Quixadá

QXD0068 - Reuso de Software

Prof. Francisco Victor da Silva Pinheiro

Equipe

Jefferson de Azevedo Maciel

Monalisa Silva Bezerra

Trabalho 1 – Refatoração com Padrões de Projeto em Sistema Legado

2026.2

Sumário

1. Introdução e descrição dos sistema **3**
1.1 Introdução 3
1.2 Descrição do Sistema 4
1.3 Componentes analisados 4
2. Estrutura e Funcionamento Geral do Sistema **5**
2.1 Organização modular 5
2.2 Editor e preferências 5
2.3 Live Development 5
3. Problemas identificados antes da refatoração **5**
3.1. confirmReload() 6
3.2. _getSelectorName 6
3.3. Função updateElementHighlightConfig() 6
3.4. Função _createRulers(editor) 6
3.5. Limites da análise inicial 6
4. Oportunidades de reutilização encontradas **6**
5. Padrões de projeto selecionados e respectivas justificativas **7**
5.1. Adapter 7
5.2. Factory 8
5.3. Builder 8
5.4. Strategy 8
6. Refatorações realizadas **8**
6.1. Refatoração com Adapter em main.js 8
6.2. Refatoração com Factory em EditorOptionHandlers.js 9
6.3. Refatoração com Builder em main.js 10
6.4. Refatoração com Strategy em CSSInLineEditor.js 10
7. Comparação entre o código antes e depois **11**
8. Instruções de Build **11**
8.1. Pré-requisitos 11
8.2 Obtenção do código-fonte 12
8.3 Instalação de dependências 12
8.4. Geração da build 12
8.5. Execução da aplicação 12
8.6. Limpeza da build 13
9. Análise dos ganhos de reutilização, manutenção e extensibilidade **13**
9.1. Coesão e acoplamento 13
9.2. Manutenibilidade, extensão e reutilização 13
9.3. Limitações 14
10. Conclusão **14**
Referências **14**

1. Introdução e descrição dos sistema

1.1 Introdução

A manutenção e a evolução de sistemas de software constituem atividades importantes no desenvolvimento de aplicações, especialmente quando o código-fonte apresenta diferentes responsabilidades, dependências entre módulos e comportamentos que precisam ser preservados durante alterações. Nesse contexto, a refatoração de software permite reorganizar estruturas internas do código sem modificar, intencionalmente, o comportamento observável do sistema. Quando aplicada de forma adequada, essa prática contribui para melhorar a legibilidade, a organização, a manutenibilidade e a capacidade de extensão do software.

Uma das estratégias utilizadas nesse processo consiste na aplicação de padrões de projeto, que apresentam soluções recorrentes para problemas comuns de organização e estruturação de sistemas orientados a objetos. A utilização desses padrões deve estar associada a problemas concretos identificados no código, de maneira que as alterações realizadas proporcionem benefícios efetivos à manutenção, à reutilização e à evolução do sistema.

1.2 Descrição do Sistema

Este trabalho utiliza como objeto de estudo o Phoenix, um editor de código de código aberto que prioriza uma experiência de desenvolvimento local simplificada, uma arquitetura leve e o suporte a tecnologias web. O Phoenix prioriza tecnologias como HTML, CSS e JavaScript, oferece edição de código baseada no CodeMirror e organiza suas funcionalidades em módulos JavaScript carregados com RequireJS. O projeto deriva do Brackets e busca oferecer um ambiente leve, com suporte a extensões compatíveis e uma experiência de desenvolvimento local simplificada.

O objetivo deste trabalho é analisar trechos selecionados do código-fonte do projeto e identificar oportunidades de melhoria por meio da aplicação de quatro padrões de projeto. A proposta consiste em realizar refatorações pontuais em funções existentes, mantendo as funcionalidades originais e buscando separar melhor as responsabilidades envolvidas. Dessa forma, pretende-se compreender como a reorganização do código pode contribuir para a manutenção e a evolução de um sistema de software existente.

1.3 Componentes analisados

Quatro arquivos foram analisados:

1. `src/main.js` , que inicializa o app e declara funções gerais.
2. `src/editor/CSSInLineEditor.js`, que gerencia como o editor se comporta ao editar CSS dentro do HTML.
3. `src/LiveDevelopment/main.js`, que integra a pré-visualização e atualiza suas configurações.
4. `src/editor/EditorOptionHandlers.js`, que gerencia preferências do editor e configura as réguas visuais. As contribuições relativas aos padrões Builder e Strategy serão complementadas.

2. Estrutura e Funcionamento Geral do Sistema

2.1 Organização modular

O Phoenix distribui as funcionalidades entre módulos responsáveis por edição, documentos, preferências, comandos, interface e pré-visualização. Os módulos declaram dependências com `define()` e `require()` e disponibilizam funcionalidades por `exports`. A comunicação ocorre por chamadas diretas e eventos, permitindo que componentes reajam a mudanças sem concentrar todo o comportamento em um único arquivo.

2.2 Editor e preferências

O componente `src/editor/Editor.js` integra o CodeMirror aos documentos e às funcionalidades específicas do Phoenix. O arquivo `EditorOptionHandlers.js` complementa esse mecanismo por meio da consulta de preferências, registro de comandos e atualização de opções como números de linha, quebra de linhas e réguas. Dessa forma, as mudanças nas preferências ou no tema podem acionar a atualização dessas opções.

2.3 Live Development

O arquivo `src/LiveDevelopment/main.js` conecta o recurso Live Development à interface. A implementação operacional está em `LiveDevMultiBrowser.js`, que resolve o documento HTML a pré-visualizar, prepara o servidor, cria os documentos acompanhados e gerencia a comunicação com o navegador. Eventos da sessão são usados para atualizar o

estado apresentado ao usuário. Essas relações são relevantes para a refatoração porque as funções alteradas dependem de contratos já utilizados por outros módulos.

3. Problemas identificados antes da refatoração

A análise inicial foi delimitada a funções dos arquivos src/LiveDevelopment/main.js e src/editor/EditorOptionHandlers.js. O objetivo não é afirmar que todo o sistema apresenta falhas estruturais, mas localizar responsabilidades que podem ser separadas de forma pontual e com baixo risco de alterar o comportamento..

3.1. confirmReload()

Em `src/main.js`, a função mistura responsabilidades de como criar um elemento e como o adicionar ao restante da tela, e não há uma separação espacial no código sobre cada elemento criado, tornando a legibilidade da função baixa, e não respeita o princípio de aberto/fechado.

3.2. _getSelectorName

Em `src/editor/CSSInLineEditor.js`, a função utiliza várias comparações condicionais para determinar o comportamento do programa, o que torna o código difícil de compreender e de adicionar funcionalidades.

3.3. Função updateElementHighlightConfig()

Em `src/LiveDevelopment/main.js`, a função consulta a preferência de destaque, aplica o valor padrão, atualiza `elemHighlights` e envia a configuração para `MultiBrowserLiveDev`. A leitura e transformação da preferência ficam misturadas à atualização da configuração, dificultando testar essa transformação isoladamente.

3.4. Função _createRulers(editor)

Em `src/editor/EditorOptionHandlers.js`, a função verifica preferências e tema, constrói cada objeto de régua dentro de um laço e aplica a lista ao CodeMirror. A criação dos objetos fica acoplada à coordenação das condições e da atualização do editor.

3.5. Limites da análise inicial

Os pontos identificados representam oportunidades de organização e testabilidade em funções específicas. Nesta etapa, não foram coletadas métricas de complexidade

ciclomática, duplicação ou acoplamento do repositório completo. Portanto, não quer dizer necessariamente que as funções originais estivessem incorretas. O critério utilizado foi a possibilidade de separar uma responsabilidade de transformação sem mudar as condições de execução já existentes.

4. Oportunidades de reutilização encontradas

No primeiro caso, a conversão da preferência para um objeto de configuração pode ser encapsulada e verificada separadamente. Se outros consumidores precisarem da mesma transformação, esse componente poderá ser reaproveitado. No segundo caso, uma fábrica centraliza a montagem dos objetos das réguas. Assim, mudanças no formato destes objetos poderão ser concentradas em um único ponto, e a transformação de colunas e cores em configurações poderá ser testada sem executar toda a rotina de atualização do editor.

5. Padrões de projeto selecionados e respectivas justificativas

Padrão	**Trecho**	**Problema e benefício esperado**
Builder	main.js `confirmReload()`	A função cria vários objetos HTML, tornando o código pouco legível. É esperado a maior legibilidade e flexibilidade do código.
Strategy	/editor/CSSInLineEditor.js `_getSelectorName()`	A função utiliza vários ifs para determinar o comportamento do programa. É esperado deixar o código mais legível e fácil e adicionar funcionalidades.
Adapter	main.js `updateElementHighlightConfig()`	Separar a conversão da preferência de destaque da atualização da configuração do Live Development.
Factory	EditorOptionHandlers.js `_createRulers(editor)`	Centralizar a criação dos objetos de configuração das réguas

5.1. Adapter

O Adapter é utilizado na proposta para converter os dados de uma preferência para a estrutura de configuração que será aplicada ao Live Development. O objeto ElementHighlightPreferenceAdapter reúne a leitura da preferência e a regra de valor padrão, retornando um objeto com a propriedade elemHighlights. A função updateElementHighlightConfig() permanece responsável por obter a configuração corrente, incorporar o resultado e chamar updateConfig().

5.2. Factory

A RulerOptionsFactory concentra a construção dos objetos que representam as réguas do CodeMirror. Seu método create() recebe as colunas, a lista de cores e a cor padrão, e retorna uma lista de objetos com as propriedades color, column e lineStyle. A função _createRulers(editor) não precisa mais definir diretamente como cada objeto é montado. A implementação utiliza uma fábrica simples, com método explícito de criação.

5.3. Builder

A classe ModalBuilder inicia a construção dos elementos utilizando um objeto de documento repassado pelo construtor, após isso é possível o usuário adicionar os elementos como título, conteúdo, botões e estilo, como desejar, após isso é possível utilizar seu método builder() para receber o objeto modal pronto.

5.4. Strategy

O arquivo SelectorStrategy.js define 4 objetos de estratégia que aplicam o contrato { isMatch, resolve }. 3 objetos de comportamento e 1 objeto de erro são implementados. A função isMatch(tokenType, attrName) decide qual condição deve ser cumprida para a seleção daquela estratégia, e a função resolve(tagInfo) é a ação a ser realizada. O módulo contido no arquivo retorna uma função que recebe tokenType e attrName e retorna a estratégia apropriada.

6. Refatorações realizadas

6.1. Refatoração com Adapter em main.js

Foi adicionado o objeto `ElementHighlightPreferenceAdapter` antes da função original. O método `toLivePreviewConfig()` concentra a leitura da preferência e a regra de valor padrão. A função mantém o fluxo de atualização da configuração:



```
const ElementHighlightPreferenceAdapter = {
    toLivePreviewConfig() {
        const prefValue = PreferencesManager.get(
            CONSTANTS.PREFERENCE_PROJECT_ELEMENT_HIGHLIGHT
        );
        return { elemHighlights: prefValue || CONSTANTS.HIGHLIGHT_HOVER };
    }
};

function updateElementHighlightConfig() {
  const config = MultiBrowserLiveDev.getConfig();
  Object.assign(config, ElementHighlightPreferenceAdapter.toLivePreviewConfig());
  MultiBrowserLiveDev.updateConfig(config);
}
```

O contrato de `updateConfig()` e as exportações do módulo não foram alterados.

6.2. Refatoração com Factory em EditorOptionHandlers.js

Foi adicionado `RulerOptionsFactory` antes de `_createRulers(editor)`. Seu método `create()` utiliza `map()` para gerar a lista de configurações:

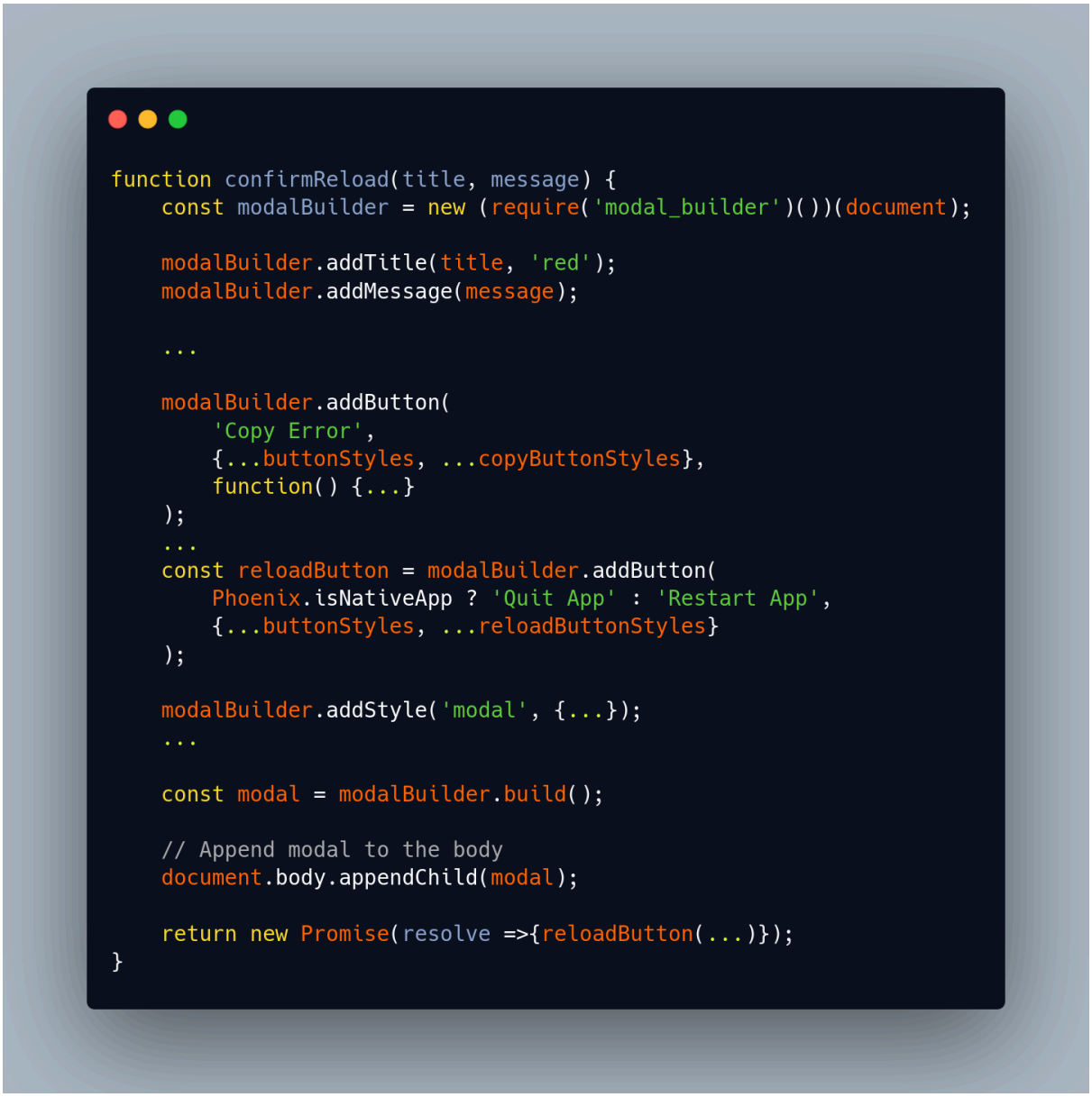


```
const RulerOptionsFactory = {
    create: function (rulerColumns, rulerColors, defaultColor) {
        return rulerColumns.map(function (column, index) {
            return {
                color: rulerColors[index] ? rulerColors[index] : defaultColor,
                column: column,
                lineStyle: "solid !important"
            };
        });
    }
};
```

Na função `_createRulers(editor)`, foram mantidas as verificações de habilitação, colunas, editor, tema e configuração já existente. O laço que montava os objetos foi substituído por uma chamada a `RulerOptionsFactory.create(...)`, cujo resultado continua sendo aplicado por `editor._codeMirror.setOption("rulers", rulerOptions)`. Os ouvintes de eventos e as preferências não foram alterados.

6.3. Refatoração com Builder em main.js

Em um arquivo a parte foi criado a classe ModalBuilder, consistindo de um método construtor que declara os elementos base para a estrutura HTML, métodos adicionais que podem adicionar elementos à estrutura geral, como título e botões, e o método build que constrói a árvore de HTML do modal e retorna a raiz para que possa ser adicionada à árvore geral.



```
function confirmReload(title, message) {
    const modalBuilder = new (require('modal_builder')())(document);

    modalBuilder.addTitle(title, 'red');
    modalBuilder.addMessage(message);

    ...

    modalBuilder.addButton(
        'Copy Error',
        {...buttonStyles, ...copyButtonStyles},
        function() {...}
    );
    ...
    const reloadButton = modalBuilder.addButton(
        Phoenix.isNativeApp ? 'Quit App' : 'Restart App',
        {...buttonStyles, ...reloadButtonStyles}
    );

    modalBuilder.addStyle('modal', {...});
    ...

    const modal = modalBuilder.build();

    // Append modal to the body
    document.body.appendChild(modal);

    return new Promise(resolve =>{reloadButton(...)});
}
```

6.4. Refatoração com Strategy em CSSInLineEditor.js

Em um arquivo a parte foi criado uma lista de objetos que seguem o template de estratégia base, o módulo retorna uma função que seleciona a estratégia apropriada. A

estrutura de condicionais que estavam na função original foram removidos e a seleção de estratégia foi adicionada.



```
function _getSelectorName(editor, pos) {
    var tagInfo = HTMLUtils.getTagInfo(editor, pos);
    const activeStrategy = require('../JSUtils/SelectorStrategy.js')(tagInfo);
    return activeStrategy.resolve(tagInfo);
}
```

7. Comparação entre o código antes e depois

Aspecto	**Antes**	**Depois**
Separação de responsabilidades	As funções liam a preferência ou montavam os objetos no mesmo fluxo que coordenava a atualização.	O Adapter encapsula a conversão. A Factory centraliza a criação das configurações.
Preservação e manutenção	Regras e verificações estavam nas funções coordenadoras.	As verificações e os formatos de saída são preservados. As regras ganham pontos definidos para revisão e testes.

Um ponto importante da comparação é que ela é qualitativa e restrita às funções modificadas. E não houve medição quantitativa de redução de complexidade e acoplamento.

8. Instruções de Build

8.1. Pré-requisitos

Para configurar o ambiente e gerar o build do projeto Phoenix, é necessário ter as seguintes ferramentas instaladas:

- Git: para obter o código-fonte do repositório.
- Node.js e npm: para gerenciar as dependências e executar o projeto.

- Gulp CLI: o projeto utiliza para automatizar as tarefas de build.

A instalação global do Gulp CLI pode ser feita pelo comando abaixo, de acordo com o sistema operacional:

- Windows: npm install -g gulp-cli
- Linux/macOS: sudo npm install -g gulp-cli

8.2 Obtenção do código-fonte

O código pode ser obtido clonando o repositório:

- git clone https://github.com/Poyo663/phoenix.git

Em seguida, deve-se acessar o diretório do projeto:

- cd phoenix

8.3 Instalação de dependências

Com o terminal aberto na raiz do projeto, execute:

- npm install

Esse comando instala as dependências necessárias para o funcionamento do projeto e executa as etapas adicionais definidas no script postinstall do package.json.

8.4. Geração da build

Após a instalação das dependências, a build pode ser gerada com o seguinte comando:

- npm run build

Esse script executa as principais etapas de preparação do projeto, incluindo a geração do visualizador de Markdown, a preparação dos arquivos necessários, a execução das tarefas de build, a geração da documentação JavaScript, a compactação dos arquivos de teste, a análise estática do código com ESLint e a verificação de vulnerabilidades nas dependências de produção.

Para gerar uma build de depuração, voltada ao desenvolvimento, pode-se utilizar:

- npm run build:debug

8.5. Execução da aplicação

Após a configuração do ambiente, a aplicação pode ser executada localmente por meio do comando:

- npm run serve

Com o servidor em execução, abra o navegador e acesse:

- http://localhost:8000/src

Por meio deste endereço a aplicação Phoenix é disponibilizada pelo servidor local.

8.6. Limpeza da build

Caso seja necessário limpar os artefatos gerados e reinicializar os arquivos de build, utilize o comando:

- npm run clean

9. Análise dos ganhos de reutilização, manutenção e extensibilidade

9.1. Coesão e acoplamento

Os objetos auxiliares concentram tarefas específicas, o adapter transforma uma preferência em configuração e a factory monta objetos de réguas. O acoplamento não desaparece, pois ambos continuam relacionados a dados ou componentes do sistema. O ganho está na separação mais explícita das responsabilidades.

9.2. Manutenibilidade, extensão e reutilização

As alterações tornam os pontos de mudança mais fáceis de localizar. Se a regra de valor padrão de elemHighlights mudar, a equipe poderá examinar o adaptador; se a estrutura de configuração das réguas mudar, poderá examinar a fábrica. Essa localização pode reduzir o esforço de compreensão em alterações futuras, principalmente quando acompanhada de testes que cubram as saídas esperadas.

A vantagem deve ser avaliada proporcionalmente ao tamanho dos trechos. As funções originais já eram relativamente curtas; portanto, não seria correto afirmar que a refatoração elimina uma complexidade elevada. O ganho esperado está na clareza semântica e na possibilidade de testar a transformação isoladamente.

A fábrica pode ser ampliada para incluir novas propriedades dos objetos de régua sem concentrar a montagem dos dados na função que gerencia o editor. O adaptador pode ser reutilizado por pontos que precisam converter a preferência de destaque no mesmo formato, caso surjam outros consumidores com essa necessidade. Esses cenários são benefícios potenciais; a existência de vários consumidores reutilizando os auxiliares precisaria ser comprovada por alterações adicionais ou por referências reais no código.

9.3. Limitações

A avaliação apresentada é qualitativa e focalizada em dois métodos. Não foram levantadas métricas antes/depois, como complexidade, cobertura, duplicação ou tempo de manutenção. Além disso, a verificação sintática não substitui testes de integração. Por essas razões, os resultados devem ser apresentados como ganhos estruturais esperados até que os testes e a revisão do código final confirmem a preservação do comportamento.

10. Conclusão

A análise do Phoenix identificou duas oportunidades localizadas de refatoração: separar a conversão de uma preferência de destaque da atualização da configuração do Live Development e centralizar a construção dos objetos utilizados nas réguas do editor. Para isso, foram propostas as aplicações dos padrões Adapter e Factory, mantendo as responsabilidades principais das funções originais.

As alterações melhoram a localização das regras e favorecem testes isolados, mas seus ganhos permanecem qualitativos até que os testes funcionais e de integração sejam executados.

Referências

Repositório oficial: https://github.com/phcode-dev/phoenix
Repositório refatorado: https://github.com/Poyo663/phoenix
https://refactoring.guru/design-patterns/builder
https://refactoring.guru/design-patterns/strategy
https://refactoring.guru/design-patterns/adapter
https://refactoring.guru/design-patterns/factory-method
https://developer.mozilla.org/en-US/docs/Web/JavaScript